MGSC 695 – Building Agentic AI Applications

Final Capstone Project - Multi-Agent Trip Planner

Group 9

Syed Shaheryar Haider – ID 261248339

Project Overview

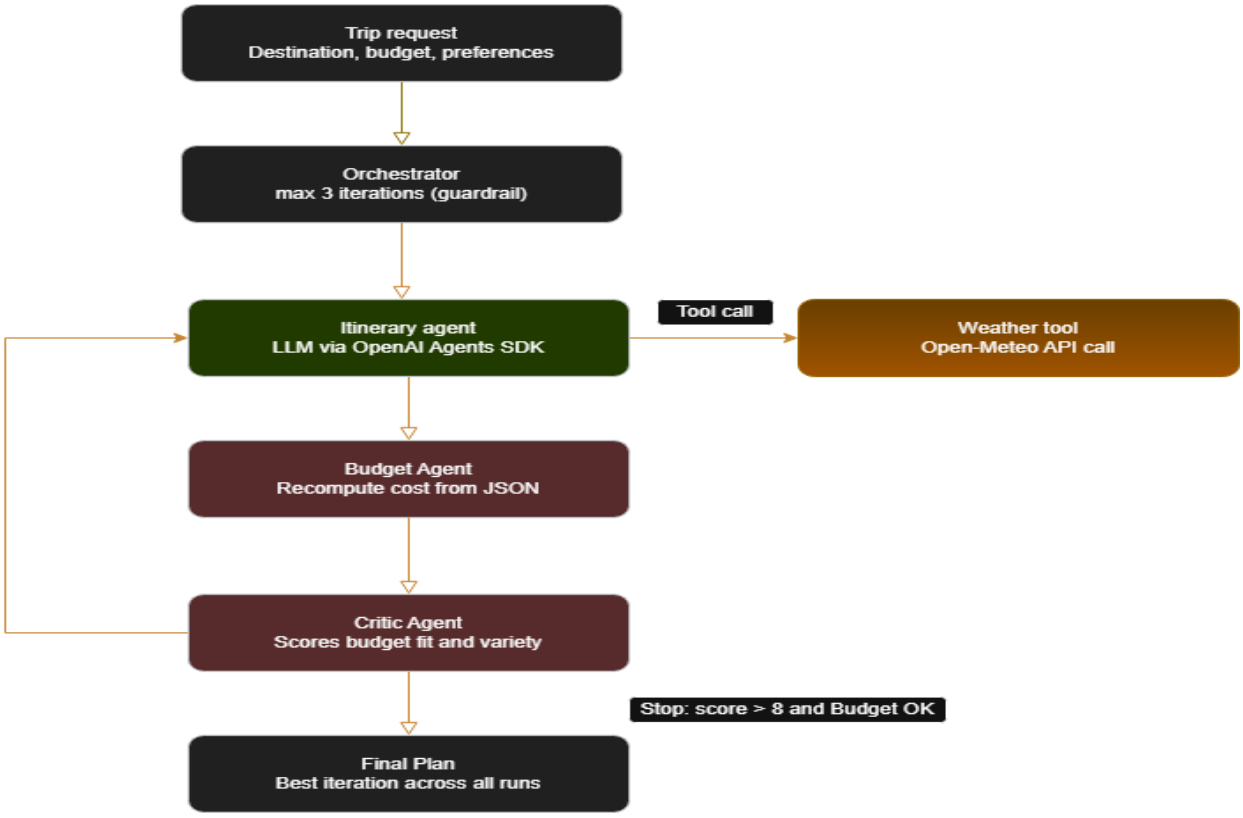
I've been planning a trip to Paris, and it occurred to me that this was the perfect opportunity to apply what we covered in this course, so I decided to build a multi-agent system that plans a real trip I'm thinking about taking.

Three agents collaborate to produce a budget-aware, weather-aware travel itinerary:

1. **Itinerary Agent** (LLM-powered): generates a day-by-day plan and calls a live weather tool to avoid scheduling outdoor activities on poor-weather days.
2. **Budget Agent** (rule-based): independently verifies the itinerary's total cost from structured data rather than trusting the LLM's self-reported arithmetic.
3. **Critic Agent** (rule-based): scores the plan on budget fit and activity variety, and feeds natural-language feedback back into the Itinerary Agent for revision.

The three agents are orchestrated in an iterative loop: the Itinerary Agent proposes a plan, the Budget Agent checks it against my budget, and the Critic Agent scores it and generates feedback that gets fed back into the next round of itinerary generation. This loop runs until the plan is judged acceptable or a fixed iteration cap is reached. I tested the system under three budget conditions (comfortable, tight, and very tight) to see how the feedback loop behaves under different constraints, partly out of curiosity about which budget actually reflects what I'd realistically spend, and partly because it turned out to be a great way to stress-test the agents against each other.

Architecture Diagram



Architecture Diagram Legend: Green box represents the LLM-powered agent, dark red boxes represent rule-based agents, the amber box represents an external tool, and white/outlined boxes represent input, orchestration, and output.

A trip request (destination, budget, preferences) flows into the Orchestrator, which manages up to 3 iterations. Each iteration, the Itinerary Agent calls the Weather tool (Open-Meteo API) to fetch live forecast data before generating a plan. The plan then passes to the Budget Agent, which recomputes the total cost from a structured JSON field in the output, then to the Critic Agent, which scores it on budget fit and variety. If the score exceeds 8 and the plan is within budget, the loop stops; otherwise, the Critic's feedback is passed back into the Itinerary Agent's next prompt (the left-side arrow). The system returns the best plan across all iterations, not simply the last one.

Description

Itinerary Agent (LLM-powered): The Itinerary Agent is the only LLM-powered component in the system. It runs on GPT-4o-mini via the OpenAI Agents SDK and is responsible for generating the day-by-day itinerary based on the user's destination, trip duration, budget, and preferences. Before generating a plan, it calls a live weather tool to fetch the forecast for the travel dates, which it factors into activity scheduling. One important design decision was requiring the agent to output a structured line at the end of every response: ACTIVITY_COSTS_JSON: [...], listing the cost of each individual activity. LLM's self-reported totals in the prose were occasionally wrong so having a structured field meant the Budget Agent could always verify the cost independently, regardless of what the LLM wrote in plain text.

Budget Agent (rule-based): The Budget Agent is entirely rule-based. It parses the ACTIVITY_COSTS_JSON array from the Itinerary Agent's output using a regex, sums the values in plain Python, and compares the result against the user's budget. It returns a verdict (WITHIN_BUDGET, OVER_BUDGET, or UNKNOWN), an estimated cost, and a short message explaining the result. Budget verification is a simple arithmetic task and delegating it to a rules-based system means the result is always deterministic and auditable.

Critic Agent (rule-based): The Critic Agent scores the itinerary out of 10: up to 5 points for budget fit (based on the Budget Agent's verdict) and up to 5 points for activity variety (a keyword count across categories like museum, market, park, tour, restaurant, etc.). It also generates a short plain-language feedback string, such as "Reduce cost to fit budget" or "Add more variety in activity types." This feedback string is appended directly to the next iteration's prompt sent to the Itinerary Agent. This is the system's learning and adaptation loop and each revision is

conditioned on the previous round's critique, so the agent is not generating from scratch each time but actively responding to prior feedback.

Orchestration and Shared State: A TripState data class serves as the shared memory across all agents. It holds the trip parameters, the current iteration count, the iteration cap (max 3), and a history list that every agent appends its output to after each step. After a full run, the history is written to a JSON log file for debugging and review. The orchestration loop tracks the best iteration seen across all rounds, ranked first by how close the plan is to budget, then by total score and returns that as the final output.

Sample Interaction

```
--- Iteration 1 | ItineraryAgent ---
[Parsed activity costs: [0, 15, 12, 25, 19, 10, 0, 15, 10, 8] -> sums to $129.00]
--- Iteration 1 | BudgetAgent ---
verdict: OVER_BUDGET — $129.00 exceeds budget by $49.00.
--- Iteration 1 | CriticAgent ---
total_score: 5 — "Reduce cost to fit budget."
--- Iteration 2 | ItineraryAgent ---
[LLM self-reported total: $73]
[Parsed activity costs: [0, 8, 0, 15, 12, 7, 0, 15, 0, 6, 10] -> sums to $73.00]
--- Iteration 2 | BudgetAgent ---
verdict: WITHIN_BUDGET — $73.00 is within the $80.00 budget.
--- Iteration 2 | CriticAgent ---
total_score: 9 — "Looks good."

Iterations used: 2 / 3
```

In iteration 1, the Itinerary Agent generated a plan that came in at $129, $49 over the $80 budget. The Budget Agent caught this by summing the structured ACTIVITY_COSTS_JSON array independently rather than trusting the LLM's own stated total. The Critic Agent scored it a 5/10 and returned the feedback "Reduce cost to fit budget," which was appended to the next prompt.

In iteration 2, the Itinerary Agent revised the plan significantly, swapping out expensive restaurants and paid attractions for cheaper alternatives like street food and free walking areas. The revised plan came in at $73, and notably the LLM's own stated total ($73) matched the parsed array exactly. The Budget Agent confirmed it was within budget, and the Critic Agent scored it 9/10, triggering the early stop condition. The system returned this plan as the final output without needing a third iteration.

Across three budget scenarios, the system behaved as follows:

Budget	Outcome	Iterations used
$600 (comfortable)	Converged on the first attempt. With a generous budget, the agent included paid museums, sit-down bistro meals, and snacks throughout the day without needing any revision.	1
$150 (tight)	Required one revision. The first attempt came in over budget, and the Critic pushed back with feedback. The second attempt landed at $144 by trimming dining costs while keeping a good mix of museums and free attractions.	2
$80 (very tight)	Required one revision. The first attempt came in at $129, well-over budget. After feedback, the agent stripped back to mostly free attractions and cheap street food, landing at $73 on the second attempt.	2

Discussion

What worked well?

The most important design decision in the project was keeping cost verification outside the LLM entirely. In one run, the Itinerary Agent wrote "Total Cost: $100" in its prose, but the ACTIVITY_COSTS_JSON array it generated summed to $75. The Budget Agent caught this every time because it never reads the LLM's stated total as it always recomputes from the structured array. Without this separation, the system would have accepted a plan it believed to be within budget based on incorrect arithmetic.

The feedback loop also worked as intended. Across multiple runs, the Itinerary Agent demonstrably changed the content of its plans in response to feedback and dropping expensive activities, substituting paid museums for free walking areas, and reducing dining budgets which shows that the agents are genuinely exchanging information and influencing each other's behavior rather than operating independently.

The weather tool integration added a practical layer of realism. In all runs, the Itinerary Agent received live forecast data showing temperatures in the high 30s Celsius for Paris, which consistently appeared in the generated plans as recommendations to stay hydrated and schedule indoor activities during the hottest parts of the day.

What was challenging?

The feedback loop does not guarantee improvement every round. During testing, one $80 run saw the cost move $129 to $92 then $99, getting worse in the final iteration despite identical feedback each time. Each revision is a fresh LLM generation responding to a short text note, not a constrained optimization. The agent can overshoot in either direction. A real bug was found due

to this during original implementation: the orchestrator was returning whichever iteration ran last, even if an earlier iteration was better. The fix was to track the best-scoring plan across all rounds and return that instead.

The Critic's variety score is a keyword count against a fixed list (museum, hike, tour, market, etc.). Itineraries that used different but reasonable nouns ("Marais District," "Seine River walk," "Picasso Museum") sometimes scored low on variety despite being genuinely varied, because the scorer does keyword matching rather than semantic understanding.

The LLM was not consistently wrong or consistently right about its self-reported totals where it got the math correct in some runs and wrong in others. I needed to enforce independent verification on every run rather than spot-checking.

Potential improvements

Future improvements could include using embeddings or an LLM to better measure itinerary variety, monitoring budget trends across iterations to prevent cost overruns, and having weather conditions directly influence activity selection rather than only providing information. The system could also be expanded with a second AI agent, such as a "local expert," to enable more advanced agent-to-agent collaboration.

Learning and Adaptation

The primary learning mechanism in the system is the feedback loop between the Critic Agent and the Itinerary Agent. After each iteration, the Critic Agent provides a score and feedback, which are included in the next prompt. As a result, the Itinerary Agent can adjust its response based on previous mistakes and improve the itinerary over time. Although the model's weights do not

change, its behavior adapts through in-context learning driven by the feedback it receives. A more advanced approach could involve storing successful itineraries in a retrieval system and using them as examples for future generations. By referencing high-scoring itineraries created for similar budgets and preferences, the Itinerary Agent could produce better plans more quickly and reduce the number of iterations needed to reach an acceptable result.

Operational considerations

Guardrails: A hard cap of 3 iterations prevents infinite loops. The weather tool has an explicit fallback to a conservative mock forecast if the Open-Meteo API is unavailable, so a network outage does not crash the run or silently degrade the output.

Logging: Every agent's output is recorded in a shared history list with the iteration number and agent role and written to a JSON file after each run.

Monitoring in production: If this system ran continuously, the key metrics to watch would be: the share of requests hitting the 3-iteration cap (rising numbers signal the feedback loop is failing to converge and prompts need tuning); the rate of UNKNOWN verdicts from the Budget Agent (signals the LLM has stopped following the structured output format); the weather API failure rate; and per-call latency and cost from the OpenAI API.

Ethical/safety considerations: The system uses live weather data from Open-Meteo to generate recommendations. If the data is unavailable or inaccurate, it defaults to assuming mild weather rather than generating potentially misleading advice. Additionally, itinerary details such as cafes, attractions, and prices are AI-generated estimates and are not independently verified. A production system should clearly inform users that these details may not be fully accurate.